# 一、摘要

本文主要针对给出的寿命数据集进行初步的数据清洗工作，并以可视化的方式展示了数据清洗阶段发现的问题以及最终清理效果。具体的数据清理范围包括但不限于对列名（数据集中各个字段的字段名）进行蛇形规范化、决定每个列的数据类型和数据范围并据此剔除逻辑异常的值、通过箱线图法查找离群值并进行合适的剔除处理，以及将缺失值用中位数或平均数替代等等。

首先，列名规范化使用了蛇形命名法，将所有列名映射为蛇形名称。然后，我通过观察数据集，规定了每列数据的数据类型，以及对应的数据范围，并将数据类型不正确或数据超出范围的行剔除。完成后，我们通过使用四分位法识别了离群值，并结合了各国的年度记录来判断异常情况。总体上，我们保留了大多数离群值，但剔除了 bmi、gdp 和 population 字段，并将部分字段中的像是缺失值的零值标记为了缺失值。最后，在缺失值处理阶段，我们删除了预期寿命与成人死亡率同时缺失的 10 条记录，并对其余的缺失值使用对应国家的平均数或中位数填补。

综上所述，原始数据由 2938 条记录和 22 个字段，调整为 2018 条记录和 19 个字段，最终数据集不再包含缺失值。本文完成了从规则检查，异常核查到缺失值处理的基本流程，为数据的后续使用提供了准备。

# 二、列名重映射

考虑到原始数据集的列名存在空格和不同的大小写，在实际编程中容易出错，因此，在读取完原始数据集后，考虑使用下表来将旧的列名（字段名）映射到已经过蛇形命名法处理过的新名称。

表 1　列名映射表

| 旧列名 | 新列名 | 旧列名 | 新列名 |
|---|---|---|---|
| Country | country | Year | year |
| Status | status | Life expectancy | life_expectancy |
| Adult Mortality | adult_mortality | infant deaths | infant_deaths |
| Alcohol | alcohol | percentage expenditure | percentage_expenditure |
| Hepatitis B | hepatitis_b | Measles | measles |
| BMI | bmi | under-five deaths | under_five_deaths |
| Polio | polio | Total expenditure | total_expenditure |
| Diphtheria | diphtheria | HIV/AIDS | hiv_aids |
| GDP | gdp | Population | population |
| thinness 1-19 years | | thinness_1_19_years | |
| thinness 5-9 years | | thinness_5_9_years | |
| Income composition of resources | | income_composition_of_resources | |
| Schooling | | schooling | |

# 三、数据类型和数据范围

通过初步观察数据集，数据准备阶段考虑按下表列出的数据类型和数据范围

检查数据集中各个字段的类型和数值范围是否正确。

表 2　数据格式用表

| 列名 | 数据类型 | 数据范围 |
|---|---|---|
| country | 枚举值(字符串) | 193 个国家 |
| year | 枚举值(整数) | [2000, 2015] |
| status | 枚举值(字符串) | {Developed, Developing} |
| life_expectancy | 浮点数 | [0,150] |
| adult_mortality | 整数 | [0, 1000] |
| infant_deaths | 整数 | [0, 1000] |
| alcohol | 浮点数 | $\geq 0$ |
| percentage_expenditure | 浮点数 | [0, 1000] |
| hepatitis_b | 整数 | [0,100] |
| measles | 整数 | [0,1000] |
| bmi | 浮点数 | $\geq 0$ |
| under_five_deaths | 整数 | [0,1000] |
| polio | 整数 | [0,100] |
| total_expenditure | 浮点数 | [0,100] |
| diphtheria | 整数 | [0,100] |
| hiv_aids | 浮点数 | [0,1000] |
| gdp | 浮点数 | $\geq 0$ |
| population | 整数 | $\geq 0$ |
| thinness_1_19_years | 浮点数 | [0,100] |
| thinness_5_9_years | 浮点数 | [0,100] |
| income_composition_of_resources | 浮点数 | [0,1] |
| schooling | 浮点数 | [0,150] |

应特别说明的是，经过检查，字段 adult_mortality（成人死亡率）和 percentage_expenditure（卫生支出占人均生产总值的百分比）中存在非常多的数据超过 100，而这些字段的单位是百分比。经利弊权衡，以防止因这些问题导致数据清洗阶段剔除太多数据，因此，这两个字段的单位在本文中按原始数据值进行解释和处理，而非视作百分比。同时，它们的数据范围从 [0,100] 被放宽到 [0,1000]。因此，adult_mortality 被解释为每 1000 人中死亡于 15 岁到 60 岁的人数。然而，对于字段 percentage_expenditure，目前只能解释为卫生指出指标，但现有的数据暂时无法解释为百分比，即便考虑将所有数据除以 100 处理，也仍然存在超过 100 的情况，也没有明确的证据可以解释为每 1000 人的支出情况。

然后，将任何不满足上表数据类型条件和值范围约束的所有行从原始数据集中剔除。这么做的原因的考量是，如果一行数据中，某个字段的数据本身存在逻辑问题（数据类型和数据范围存在错误），那么这一行的其他字段的可信度则可能存疑。为了安全起见，以尽可能避免存在逻辑错误的数据被引入数据分析过程，该行的其他字段即便满足要求，也不被信任，故而整行数据丢弃。

按上表规则对数据集进行首轮剔除后，被剔除的字段对应的行数如下所示。本轮次剔除从原始数据集的 2938 行中剔除了 910 行，剩余 2028 行。当然，这个过程中，不把任何缺失值视作应当被剔除的对象，因为之后我们将考虑使用平均

数、中位数等方案对缺失数据进行填充。

表 3　　首轮次剔除的字段行数

| 字段名 | 剔除原因 | 剔除行数 |
|---|---|---|
| percentage_expenditure | 值超出范围 | 413 |
| measles | 值超出范围 | 525 |
| under_five_deaths | 值超出范围 | 16 |
| infant_deaths | 值超出范围 | 13 |

在进行数据剔除时，发现一些数据中的 population（人口）字段不是整数。进一步研究发现，这些不是整数的数据都来自于南非，而剔除这些行会导致南非的大部分数据丢失，因此，这部分非整数数据考虑使用四舍五入到整数进行处理，对应行没有被直接移除。

# 四、离群值的处理

不考虑缺失值，将剔除了逻辑数据的数据集纳入评估，并逐列按 IQR 相关公式，绘制了每个字段的数据的箱线图，如图 1、图 2 和图 3 所示。

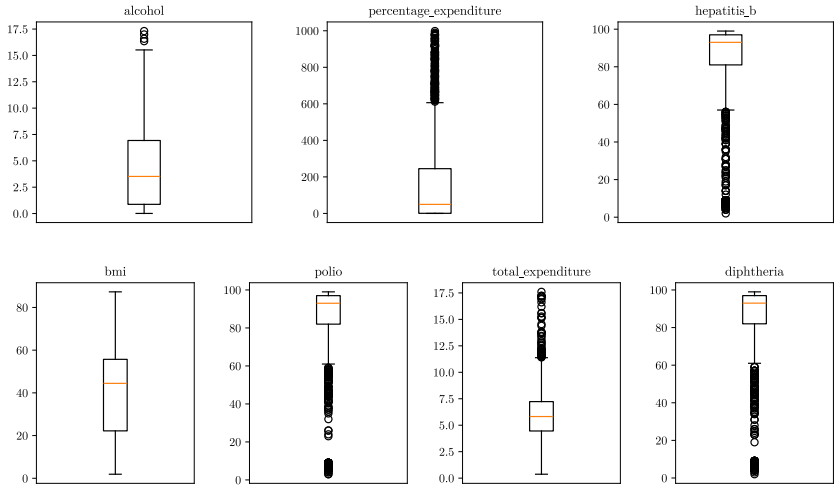



图 1　　健康行为与卫生服务相关字段的箱线图

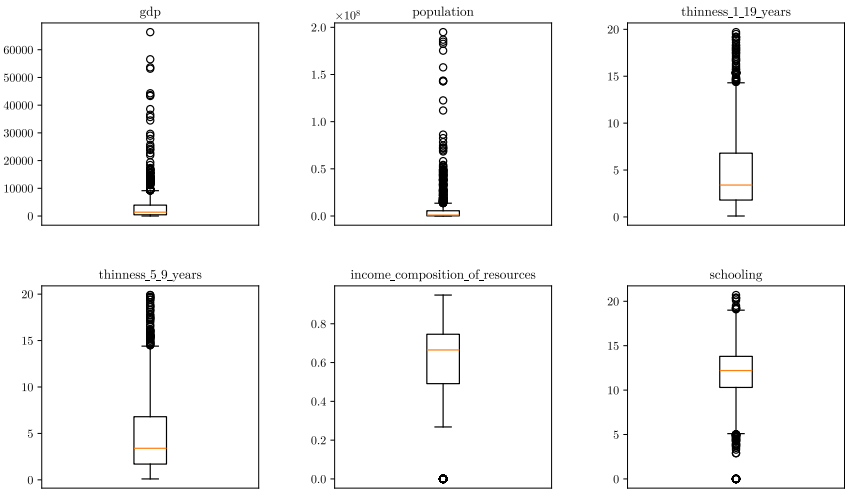



图 2　社会经济与营养相关字段的箱线图

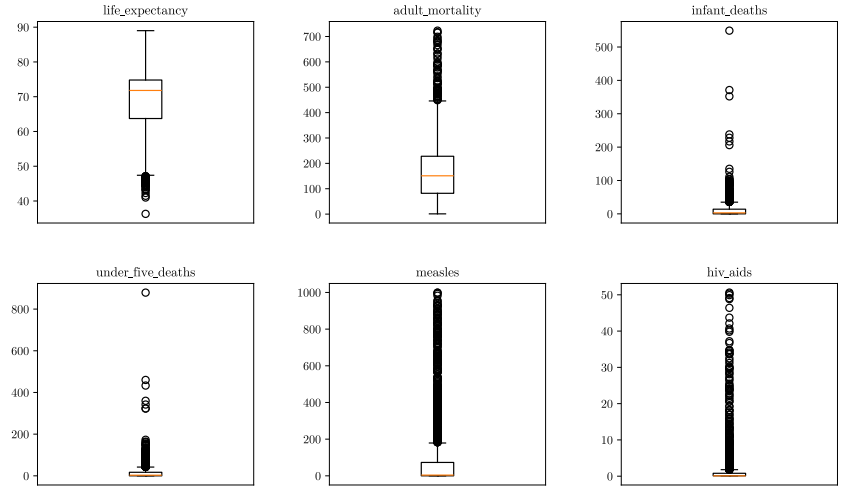



图 3　寿命与疾病相关字段的箱线图

下面图 4 的条形统计图则展示了各个字段中离群值和正常值的占比情况，占比通过堆叠的方式进行体现。从上到下依次是离群值最多的字段，到离群值最少的字段。简单观察可以发现，字段 hiv_aids（每一千名婴儿中死于艾滋病的人数）的离群值数量最多，高达 376 条，其次是 measles（每一千人中报告的麻疹病例数）达 337 条。从后往前看，bmi（体重指数）字段则不存在任何离群值，其次是 alcohol（饮酒量）字段，它仅仅具有 4 个离群值。

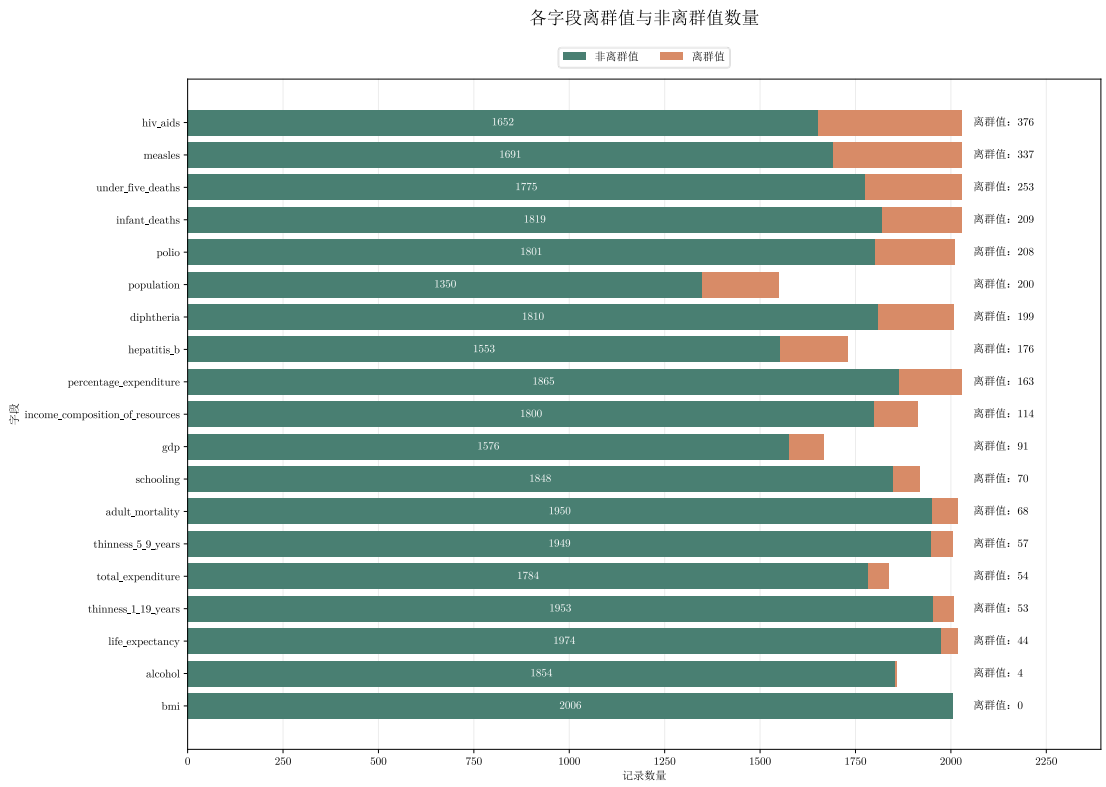



图 4　　各字段离群值和正常值的数量情况

经过评估，大多数字段的离群值考虑保留，不进行替换或移除。这么做的目的是考虑到不同国家之间的数据可能存在较大的差异，所以本文中予以容忍，仅进行相对基础的数据清理。下面列出了需要进行异常值处理的字段，以及相应的处理方法。

bmi（体重指数）字段是需要关注的对象，下面的折线统计图（图 5）分别考察了美国、海地、塞拉利昂和越南这些国家自 2000 年至 2015 年间的 bmi 变化趋势。可以很明显的观察出，美国、海地和塞拉利昂这些国家的 bmi 在某些年份存在骤降然后骤升的现象，不符合正常变化规律。对于越南，数据存在不合理的骤升趋势。除此外，塞拉利昂和越南的部分数据缺失。因此，bmi 字段的可信任性存在较大的问题，其中存在了一些无法被解释的数据错误，故本文中不考虑将 bmi 字段纳入最终表，而是考虑在数据清理阶段，将 bmi 列从数据集中剔除，但原始数据集仍然保留它。

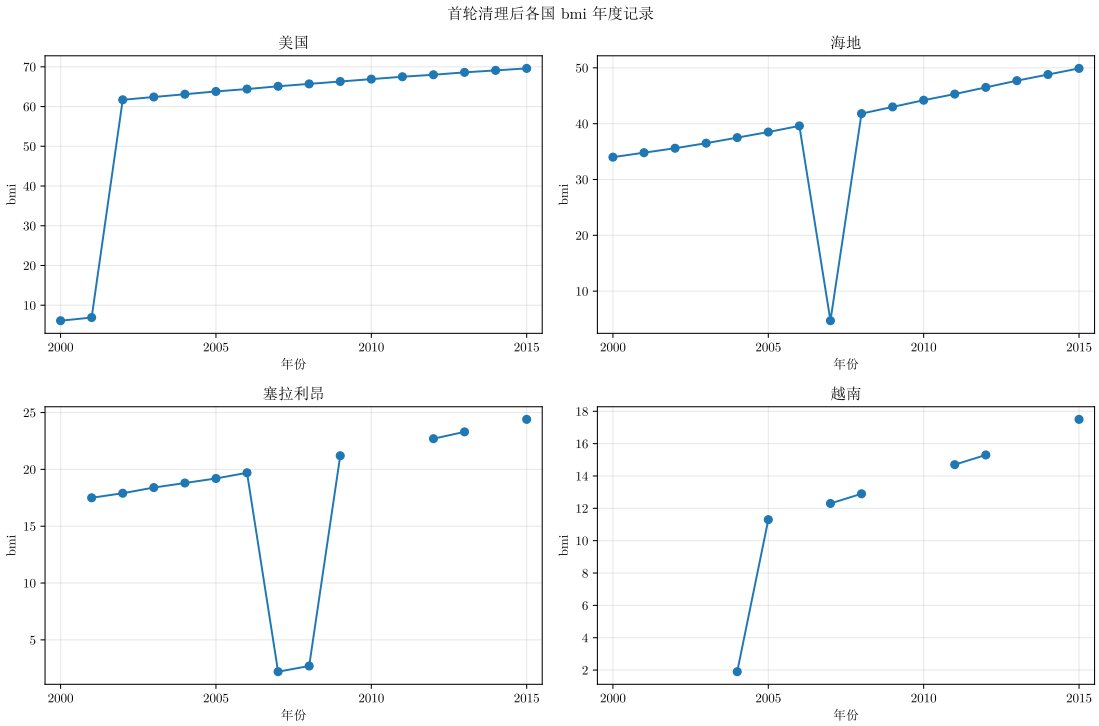



图 5　多个国家的 bmi 逐年变化情况

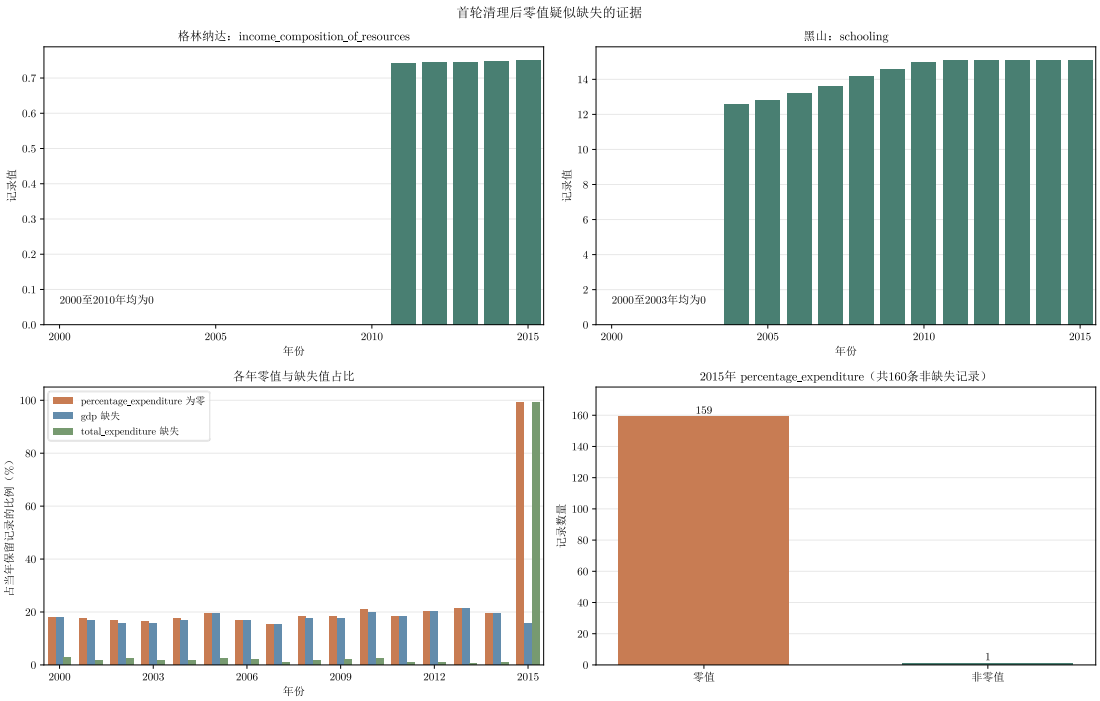



图 6　零值字段疑似是缺失值的证据

如图 6，柱状统计图展示了部分国家在字段 income_composition_of_resources（人类发展指数）, schooling（平均教育年限）和 percentage_expenditure（卫生支出人均生产总值的占比）的相关记录，包括零值与其他字段缺失的分布情况。可以发现，格林纳达的 income_composition_of_resources 在 2010 年以前都为 0

值，然后之后呈现较高水平，黑山的 schooling 数据也存在相似的问题。另外，对于 percentage_expenditure 字段，在 2015 年中的 160 条记录中，有 159 条数据呈现 0 值，意味着几乎整年的记录都是零值，并且可以从图中观察到，它和 total_expenditure 的缺失数量相近，而其他年份中，和 gdp 的缺失数量相近。种种迹象表明，percentage_expenditure 字段的 0 值很可能是缺失值，而不是 0 值。综合上面的证据，本文将字段 income_composition_of_resources, schooling 和 percentage_expenditure 中的零值视作缺失值，而非存在数据的值，并且目前不考虑剔除这些数据的所在行。

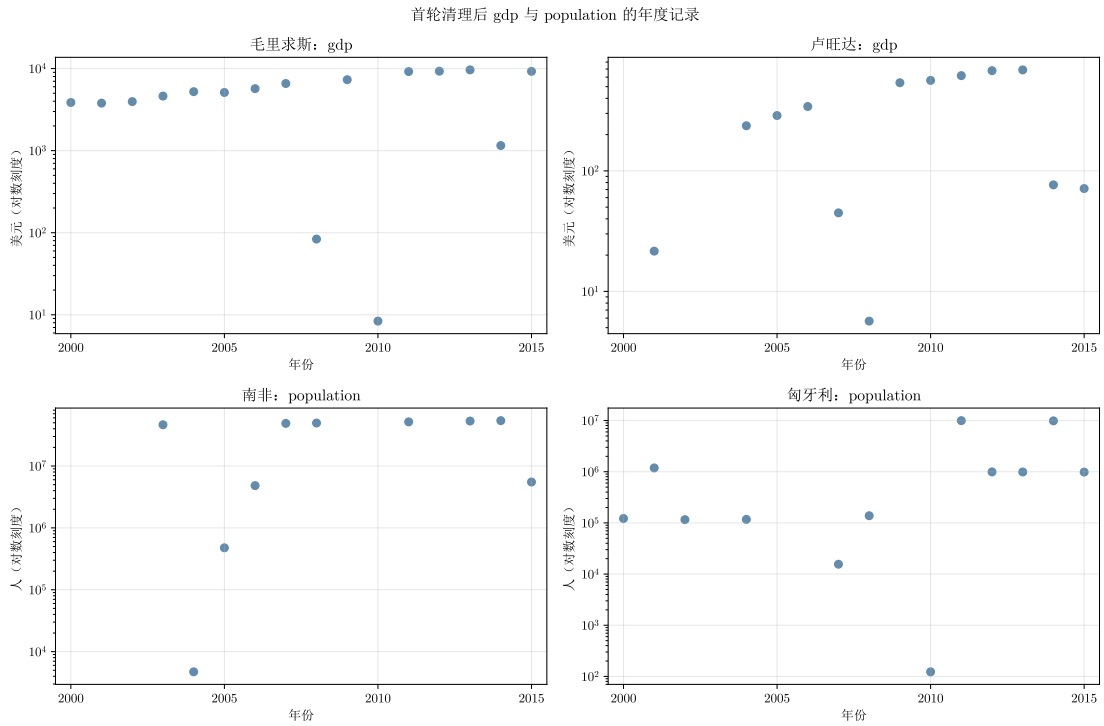



图 7　数量级变化异常的证据

gdp（生产总值）和 population（人口）字段也存在一些疑点，具体表现为数量级上的变异。图 7 中四个子图都采用了对数刻度，可以明显发现，毛里求斯和卢旺达的 GDP 水平在部分年份出现数量级骤降的现象，随后有回升。同时，南非和匈牙利的人口数据在部分年份呈现多个数量级的下降，即，从数千万或数百万人降至数千甚至数百人的记录，随后又重新上升回正常水平。这些迹象表明，如上的这些 GDP 或人口变化难以通过正常的经济发展和人口变动解释，说明数据中可能存在数量级上的录入错误。因此，为了减少这部分数据造成的分析误差，列 gdp 和 population 不考虑计入清理后的数据集，在本轮次清理中被剔除，但原始数据集中保留。

# 五、缺失值的处理

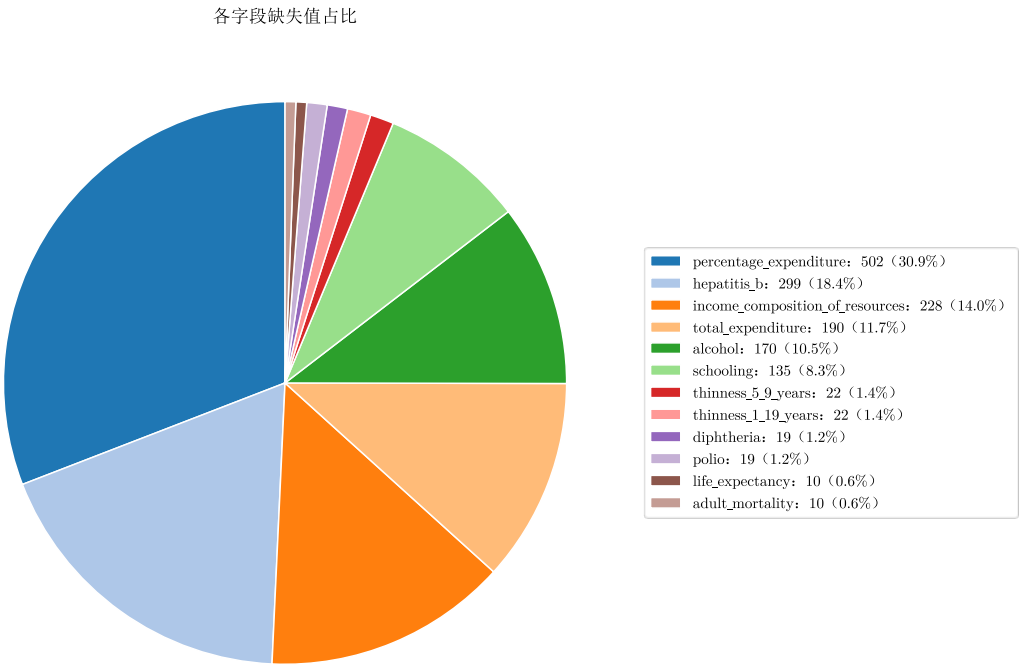



图 8　　各个字段缺失值占比情况

如图 8，这个饼状图展示了在第二轮次数据清洗后，剩余数据中，各个字段的缺失值的分布情况。该饼图覆盖了总共 1626 个缺失值，其中，可以发现字段 percentage_expenditure（卫生支出人均生产总值的占比）, hepatitis_b（1 岁儿童乙型肝炎免疫接种的覆盖率）和 income_composition_of_resources（人类发展指数）存在的缺失值最多，分别占比 30.9%, 18.4%和 14.0%。这意味着大部分缺失值都只存在于少数的字段，对于剩余的字段，缺失值较少。

经过评估，为了简单和易于理解，考虑针对每个字段使用下表给出的处理办法来保留、替换或丢弃缺失值。

表 4　　各字段缺失值处理办法

| 字段名 | 字段意义（简略） | 缺失量 | 处理方式 |
|---|---|---|---|
| life_expectancy | 预期寿命 | 10 | 丢弃对应行 |
| adult_mortality | 成人死亡率 | 10 | 丢弃所在行 |
| alcohol | 饮酒量 | 170 | 同国家中位数填补 |
| percentage_expenditure | 卫生支出占比 | 502 | 同国家中位数填补 |
| hepatitis_b | 儿童乙型肝炎接种率 | 299 | 同国家中位数填补 |
| polio | 儿童脊髓灰质炎接种率 | 19 | 同国家中位数填补 |
| total_expenditure | 政府卫生支出占比 | 190 | 同国家中位数填补 |
| diphtheria | 儿童白喉破伤风类接种 | 19 | 同国家中位数填补 |
| thinness_1_19_years | 儿童和青少年的瘦弱率 | 22 | 同国家中位数填补 |
| thinness_5_9_years | 儿童的瘦弱率 | 22 | 同国家中位数填补 |
| income_composition_of_resources | 人类发展指数 | 228 | 同国家平均数填补 |
| schooling | 平均教育年限 | 135 | 同国家平均数填补 |

对于表 4 前两项，选择丢弃的原因是因为这些字段的缺失值数量较少，并且这两个字段的缺失值都占有了相同的 10 行。从数据集中丢弃这 10 行是可行的，也解决了这两个字段的缺失值问题。对于 schooling（受教育年限），考虑到同一国家的有效记录通常变化较平缓，使用平均数，可以代表该国已有记录的平均教育水平。对于 income_composition_of_resources（发展指数），将疑似缺失的零值排除后，同一国家的有效记录通常比较接近，平均数可以概括其总体水平。对于剩余的存在缺失值的字段，卫生支出、饮酒量、消瘦比例等可能存在偏高的记录，而接种率则可能存在异常偏低的记录。这些数值容易拉高或拉低平均数，然而中位数受到的影响较小，因此使用中位数可以减少过大或过小值引起的扰动。

按表 4 的方式进行缺失值处理后，各个字段的缺失值的处理情况如下图 9 所示。当然，如果某个国家都没有数据，则考虑使用同类国家的平均值或中位数进行填补。通过观察下图，可以发现，卫生支出指标的缺失量是最多的，并且更多地依赖来自于同类国家的数据来填补和替换缺失值。综上所述，共使用本国统计量填补 899 个值，同类国家统计量填补 685 个值，另有 42 个缺失值随 10 行记录排除。全部处理完毕后，最终保留了 2018 行记录，所得数据集不再包含缺失值。

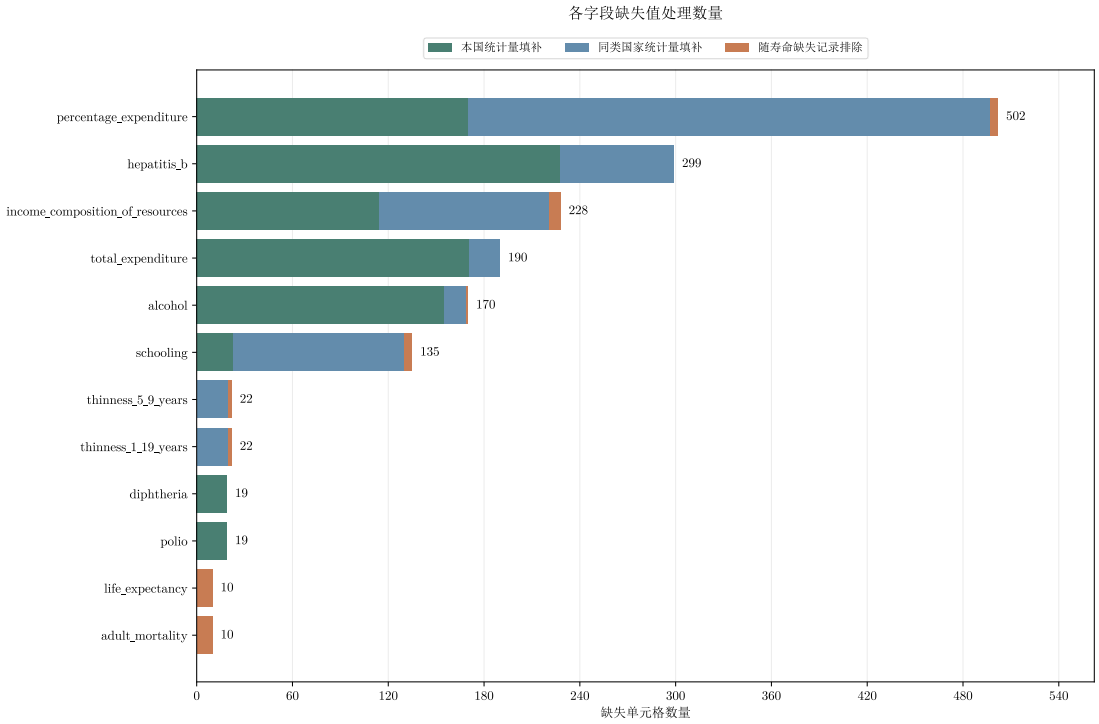



图 9　　各字段缺失值处理情况

# 六、结论

本次作业围绕寿命数据集完成了列名规范化、数据类型与范围检查、离群值核查以及缺失值处理。第一轮次的数据清理围绕数据类型和数据范围，剔除了 910 条记录，余下 2028 条记录。第二轮则结合年度变化情况，删除 bmi、gdp 和 population 三个字段，并将部分疑似缺失值但是为零值的字段标记为缺失值。最后，排除了预期寿命和成人死亡率同时缺失的 10 条记录，其他字段采用对应国

家的中位数或平均值来填补缺失值，填补来源采用对应的国家。

通过清洗过程可以发现，仅凭数值是否超出箱线图边界，难以判断数据是否错误。部分离群值可能反映国家之间的实际差异，因此本文没有将其全部删除。相反，bmi 虽然未被四分位距法识别出离群值，但在部分国家的年度记录中存在明显突变，说明统计筛查还需要结合字段含义和记录之间的连续性。

当然，本文在识别一些零值的数据时，巧妙地将这些数据和其他字段，以时间为关系的角度关联了起来，发现了一部分零值很可能是缺失值。然而，采用中位数或平均数来填补缺失值的办法可能存在不足，也许应该使用真实的数据进行填补，或考虑使用线性插值。不过，平均数或中位数法操作起来简单，也容易解释。

# 七、附录 A

基础清洗与统计部分

```
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator
from pathlib import Path
from typing import Any

class ColumnConfig:
    COLUMN_MAPPING: dict[str, str] = {
        "Country": "country",
        "Year": "year",
        "Status": "status",
        "Life expectancy": "life_expectancy",
        "Adult Mortality": "adult_mortality",
        "infant deaths": "infant_deaths",
        "Alcohol": "alcohol",
        "percentage expenditure": "percentage_expenditure",
        "Hepatitis B": "hepatitis_b",
        "Measles": "measles",
        "BMI": "bmi",
        "under-five deaths": "under_five_deaths",
        "Polio": "polio",
        "Total expenditure": "total_expenditure",
        "Diphtheria": "diphtheria",
        "HIV/AIDS": "hiv_aids",
        "GDP": "gdp",
        "Population": "population",
        "thinness  1-19 years": "thinness_1_19_years",
        "thinness 5-9 years": "thinness_5_9_years",
                    "Income    composition    of    resources":
"income_composition_of_resources",
        "Schooling": "schooling",
    }
    INTEGER_COLUMNS: tuple[str, ...] = (
        "year",
```

```
        "adult_mortality",
        "infant_deaths",
        "hepatitis_b",
        "measles",
        "under_five_deaths",
        "polio",
        "diphtheria",
        "population",
    )
    ALL_COLUMNS: tuple[str, ...] = tuple(COLUMN_MAPPING.values())
    ZERO_AS_MISSING_COLUMNS: tuple[str, ...] = (
                "income_composition_of_resources",   "schooling",
"percentage_expenditure",
    )
       SECOND_ROUND_DROP_COLUMNS:   tuple[str,   ...]   =   ("gdp",
"population", "bmi")

class ValidationRules:
    STATUS_VALUES: set[str] = {"Developed", "Developing"}
    VALUE_RANGES: dict[str, tuple[int, int | None]] = {
        "year": (2000, 2015),
        "life_expectancy": (0, 150),
        "adult_mortality": (0, 1000),
        "infant_deaths": (0, 1000),
        "alcohol": (0, None),
        "percentage_expenditure": (0, 1000),
        "hepatitis_b": (0, 100),
        "measles": (0, 1000),
        "bmi": (0, None),
        "under_five_deaths": (0, 1000),
        "polio": (0, 100),
        "total_expenditure": (0, 100),
        "diphtheria": (0, 100),
        "hiv_aids": (0, 1000),
        "gdp": (0, None),
        "population": (0, None),
        "thinness_1_19_years": (0, 100),
        "thinness_5_9_years": (0, 100),
        "income_composition_of_resources": (0, 1),
        "schooling": (0, 150),
    }

class PlotConfig:
    BOX_GROUPS: dict[str, tuple[str, ...]] = {
        "寿命与疾病": (
            "life_expectancy", "adult_mortality", "infant_deaths",
            "under_five_deaths", "measles", "hiv_aids",
        ),
        "健康行为与卫生服务": (
             "alcohol", "percentage_expenditure", "hepatitis_b",
"bmi",
            "polio", "total_expenditure", "diphtheria",
        ),
        "社会经济与营养": (
                    "gdp",  "population",  "thinness_1_19_years",
"thinness_5_9_years",
```

```
            "income_composition_of_resources", "schooling",
        ),
    }
    STYLE: dict[str, Any] = {
        "font.family": ["Latin Modern Math", "SimSun"],
        "mathtext.fontset": "cm",
        "axes.formatter.use_mathtext": True,
        "axes.unicode_minus": False,
        "svg.fonttype": "path",
    }

def load_data(path: str | Path | None = None) -> pd.DataFrame:
    if path is None:
        path = Path(__file__).with_name("寿命数据.csv")
    df = pd.read_csv(path)
    df.columns = df.columns.str.strip()
    return df.rename(columns=ColumnConfig.COLUMN_MAPPING)

def is_missing(value: object) -> bool:
    if pd.isna(value):
        return True
    if isinstance(value, str) and value.strip() == "":
        return True
    return False

def check_value(column: str, value: Any) -> tuple[Any, bool]:
    if is_missing(value):
        return value, True
    if column == "country":
        return value, isinstance(value, str)
    if column == "status":
            return value, isinstance(value, str) and value in
ValidationRules.STATUS_VALUES
    if isinstance(value, (bool, np.bool_)):
        return value, False
    number = pd.to_numeric(value, errors="coerce")
    if pd.isna(number) or not np.isfinite(number):
        return value, False

    if column == "population" and number >= 0 and number % 1 !=
0:
        integer_part = math.floor(number)
        if number - integer_part >= 0.5:
            integer_part += 1
        number = integer_part
        value = number

    if column in ColumnConfig.INTEGER_COLUMNS and number % 1 !=
0:
        return value, False
    lower, upper = ValidationRules.VALUE_RANGES[column]
    return value, bool(number >= lower and (upper is None or number
<= upper))

def drop_problem_rows(df: pd.DataFrame) -> tuple[pd.DataFrame,
dict[str, int]]:
```

```
    for column in ColumnConfig.ALL_COLUMNS:
        if column not in df.columns:
            raise ValueError(f"缺少必需列：{column}")

    prepared_df = df.copy()
    population_column = df.columns.get_loc("population")
    keep_positions: list[int] = []
     removed_counts: dict[str, int] = {column: 0 for column in
ColumnConfig.ALL_COLUMNS}

    for position in range(len(df)):
        row = df.iloc[position]
        row_has_problem = False
        for column in ColumnConfig.ALL_COLUMNS:
            value = row[column]
            if is_missing(value):
                continue
            checked_value, valid = check_value(column, value)
            if column == "population" and checked_value != value:
                  prepared_df.iloc[position, population_column] =
checked_value
            if not valid:
                removed_counts[column] += 1
                row_has_problem = True
        if not row_has_problem:
            keep_positions.append(position)

              return      prepared_df.iloc[keep_positions].copy(),
removed_counts

def find_outliers_iqr(df: pd.DataFrame) -> pd.DataFrame:
    """对清理后的数值指标逐列忽略缺失值，统计离群值。"""
    results: list[dict[str, str | int | float]] = []
    for column in ValidationRules.VALUE_RANGES:
        if column == "year":
            continue
                        values     =     pd.to_numeric(df[column],
errors="coerce").dropna()
          q1  =  float(values.quantile(0.25))  if  len(values)  else
np.nan
          q3  =  float(values.quantile(0.75))  if  len(values)  else
np.nan
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr

        outlier_count = 0
        for value in values:
            if value < lower or value > upper:
                outlier_count += 1
           percentage  =  outlier_count  /  len(values)  *  100  if
len(values) else np.nan
        results.append(
            {
                "字段": column,
                "非缺失数量": len(values),
```

```
                "下四分位数": q1,
                "上四分位数": q3,
                "四分位距": iqr,
                "下界": lower,
                "上界": upper,
                "离群值数量": outlier_count,
                "离群值比例（%）": percentage,
            }
        )
    return pd.DataFrame(results)

def plot_boxplot_group(
    df: pd.DataFrame, columns: tuple[str, ...], output_path: Path
) -> None:
    """六个字段排成两行各三个，七个字段排成上三下四。"""
    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(12, 7))
        for index, column in enumerate(columns):
            if len(columns) == 7 and index >= 3:
                plt.subplot(2, 4, index + 2)
            else:
                plt.subplot(2, 3, index + 1)
                        values = pd.to_numeric(df[column],
errors="coerce").dropna()
            plt.boxplot(values)
            plt.title(column, fontsize=11)
            plt.xticks([])
        plt.subplots_adjust(wspace=0.35, hspace=0.3)
                    plt.savefig(output_path, format="svg",
bbox_inches="tight")
        plt.close()

def plot_outlier_counts(df: pd.DataFrame, output_path: Path) ->
None:
    """按离群值数量降序绘制堆叠条形图，缺失值不参与统计。"""
    report = find_outliers_iqr(df)
    report = report.sort_values("离群值数量", ascending=False)
    outlier_counts = report["离群值数量"]
    non_outlier_counts = report["非缺失数量"] - outlier_counts

    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(14, 10))
                plt.barh(report[" 字 段 "], non_outlier_counts,
color="#497f72", label="非离群值")
                plt.barh(report[" 字 段 "], outlier_counts,
left=non_outlier_counts, color="#d88b67", label="离群值")
                for position, (non_outlier, outlier) in
enumerate(zip(non_outlier_counts, outlier_counts)):
            if non_outlier:
                        plt.text(non_outlier / 2, position,
str(non_outlier), ha="center", va="center", color="white")
            plt.text(len(df) + max(len(df), 1) * 0.015, position,
f"离群值: {outlier}", va="center")
        plt.gca().invert_yaxis()
```

```
        plt.title("各字段离群值与非离群值数量", fontsize=16, pad=50)
        plt.xlabel("记录数量")
        plt.ylabel("字段")
        plt.legend(loc="lower center", bbox_to_anchor=(0.5, 1.01),
ncol=2)
        plt.grid(axis="x", alpha=0.25)
        plt.gca().set_axisbelow(True)
        plt.xlim(0, max(len(df), 1) * 1.18)
        plt.gca().xaxis.set_major_locator(MaxNLocator(integer=Tru
e))
        plt.tight_layout()
                        plt.savefig(output_path,     format="svg",
bbox_inches="tight")
        plt.close()

def plot_missing_pie(df: pd.DataFrame, output_path: Path) -> None:
    """对第二轮处理后的数据绘制各字段缺失数量的占比。"""
    missing_counts = df.isna().sum()
         missing_counts    =    missing_counts[missing_counts    >
0].sort_values(ascending=False)
    total_missing = int(missing_counts.sum())
    if total_missing == 0:
        print("当前数据没有缺失值，无需绘制饼图。")
        return

    labels: list[str] = []
    for column, count in missing_counts.items():
        percentage = count / total_missing * 100
        labels.append(f"{column}：{count}（{percentage:.1f}%）")

    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(12, 7))
                        plt.pie(missing_counts,     startangle=90,
colors=plt.cm.tab20.colors,
               wedgeprops={"edgecolor": "white"})
        plt.title("各字段缺失值占比")
        plt.legend(labels, loc="center left",
                   bbox_to_anchor=(1, 0.5), fontsize=10)
        plt.tight_layout()
                        plt.savefig(output_path,     format="svg",
bbox_inches="tight")
        plt.close()

def  plot_missing_fill_counts(report:  pd.DataFrame,  output_path:
Path) -> None:
    """用堆叠条形图展示每个字段的缺失值如何处理。"""
    report = report.sort_values("处理前缺失", ascending=False)
    country_counts = report["同国填补"]
    status_counts = report["同类国家填补"]
    removed_counts = report["随记录排除"]
    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(12, 8))
         plt.barh(report["字段"], country_counts, color="#497f72",
label="本国统计量填补")
```

```
                plt.barh(report["  字  段  "],   status_counts,
left=country_counts,
                color="#638cac", label="同类国家统计量填补")
                 plt.barh(report["  字  段  "],   removed_counts,
left=country_counts + status_counts,
                color="#c87c53", label="随寿命缺失记录排除")
        for position, count in enumerate(report["处理前缺失"]):
            plt.text(count + 5, position, str(count), va="center")
        plt.gca().invert_yaxis()
        plt.title("各字段缺失值处理数量", pad=40)
        plt.xlabel("缺失单元格数量")
        plt.legend(loc="lower center", bbox_to_anchor=(0.5, 1.01),
ncol=3, fontsize=9)
        plt.grid(axis="x", alpha=0.25)
        plt.gca().set_axisbelow(True)
        plt.xlim(0, report["处理前缺失"].max() * 1.12)
        plt.gca().xaxis.set_major_locator(MaxNLocator(integer=Tru
e))
        plt.tight_layout()
                        plt.savefig(output_path,     format="svg",
bbox_inches="tight")
        plt.close()

def plot_data_quality(
    df: pd.DataFrame, output_dir: str | Path | None = None
) -> tuple[list[Path], Path]:
    """保存三张分组箱线图及离群值数量堆叠图。"""
    output_dir = Path(output_dir) if output_dir is not None else
Path(__file__).resolve().with_name("figures")
    boxplot_dir = output_dir / "箱线图"
    boxplot_dir.mkdir(parents=True, exist_ok=True)
    boxplot_paths: list[Path] = []
    for title, columns in PlotConfig.BOX_GROUPS.items():
        path = boxplot_dir / f"{title}.svg"
        plot_boxplot_group(df, columns, path)
        boxplot_paths.append(path)
    outlier_path = output_dir / "离群值数量.svg"
    plot_outlier_counts(df, outlier_path)
    return boxplot_paths, outlier_path

if __name__ == "__main__":
    original_df = load_data()
    df, report = drop_problem_rows(original_df)
    print(f"原始行数：{len(original_df)}")
    print(f"删除行数：{len(original_df) - len(df)}")
    print(f"剩余行数：{len(df)}")
    print("各字段不合格行数：")
    for column, count in report.items():
        print(f"{column}：{count}")

    outlier_report = find_outliers_iqr(df)
    print("\n 各字段离群值统计：")
    print(
        outlier_report.to_string(
```

```
            index=False, float_format="{:.2f}".format, na_rep="无
数据"
        )
    )

    boxplot_paths, outlier_path = plot_data_quality(df)
    print("\n三张分组箱线图已保存：")
    for path in boxplot_paths:
        print(path)
    print(f"离群值数量图已保存：{outlier_path}")
```

# 八、附录 B

异常处理与补全部分

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from prepare import (
    ColumnConfig, PlotConfig, drop_problem_rows, load_data,
    plot_missing_pie, plot_missing_fill_counts,
)

class FillConfig:
    METHODS: dict[str, str] = {
        "alcohol": "median",
        "percentage_expenditure": "median",
        "hepatitis_b": "median",
        "polio": "median",
        "total_expenditure": "median",
        "diphtheria": "median",
        "thinness_1_19_years": "median",
        "thinness_5_9_years": "median",
        "income_composition_of_resources": "mean",
        "schooling": "mean",
    }

def clean_second_round(df: pd.DataFrame) -> pd.DataFrame:
    """删除三个不可靠字段，并将指定字段的零值替换为缺失值。"""
    cleaned_df =
df.drop(columns=list(ColumnConfig.SECOND_ROUND_DROP_COLUMNS)).co
py()
    for column in ColumnConfig.ZERO_AS_MISSING_COLUMNS:
        cleaned_df[column] = cleaned_df[column].replace(0,
np.nan)
    return cleaned_df

def fill_missing_values(df: pd.DataFrame) -> tuple[pd.DataFrame,
pd.DataFrame]:
    """排除寿命缺失行，再按国家、status 的统计量补全分析数据。"""
    filled_df = df.dropna(subset=["life_expectancy"]).copy()
    report = df.isna().sum().to_frame("处理前缺失")
```

```
    report.index.name = "字段"
    report["随记录排除"] = report["处理前缺失"] -
filled_df.isna().sum()
    report["同国填补"] = 0
    report["同类国家填补"] = 0

    for column, method in FillConfig.METHODS.items():
        missing = filled_df[column].isna()
        # 两种统计量都来自填补前的有效记录。
        country_values =
filled_df.groupby("country")[column].transform(method)
        status_values =
filled_df.groupby("status")[column].transform(method)
        filled_df[column] =
filled_df[column].fillna(country_values)
        remaining = int(filled_df[column].isna().sum())
        report.loc[column, "同国填补"] = int(missing.sum()) -
remaining
        filled_df[column] =
filled_df[column].fillna(status_values)
        report.loc[column, "同类国家填补"] = remaining -
int(filled_df[column].isna().sum())
        if column in ColumnConfig.INTEGER_COLUMNS:
            filled_df.loc[missing, column] =
np.floor(filled_df.loc[missing, column] + 0.5)

    report["剩余缺失"] = filled_df.isna().sum()
    report = report[report["处理前缺失"] > 0].reset_index()
    return filled_df, report

def plot_bmi_lines(df: pd.DataFrame, output_path: Path) -> None:
    """用四个国家的年度折线展示 bmi 数值的异常变化。"""
    countries = {
        "United States of America": "美国",
        "Haiti": "海地",
        "Sierra Leone": "塞拉利昂",
        "Viet Nam": "越南",
    }
    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(12, 8))
        for index, (country, name) in
enumerate(countries.items(), start=1):
            country_data = df[df["country"] ==
country].sort_values("year")
            # 被剔除的年份留为空值，折线在这些位置自动断开。
            bmi =
country_data.set_index("year")["bmi"].reindex(range(2000, 2016))
            plt.subplot(2, 2, index)
            plt.plot(bmi.index, bmi, marker="o")
            plt.title(name)
            plt.xlabel("年份")
            plt.ylabel("bmi")
            plt.xticks([2000, 2005, 2010, 2015])
            plt.xlim(1999.5, 2015.5)
            plt.grid(alpha=0.3)
```

```
        plt.suptitle("首轮清理后各国 bmi 年度记录")
        plt.tight_layout()
        plt.savefig(output_path, format="svg",
bbox_inches="tight")
        plt.close()

def plot_zero_values(df: pd.DataFrame, output_path: Path) ->
None:
    """用四张柱状子图展示零值疑似缺失的依据。"""
    examples = [
        ("income_composition_of_resources", "Grenada", "格林纳达",
"2000 至 2010 年均为 0"),
        ("schooling", "Montenegro", "黑山", "2000 至 2003 年均为
0"),
    ]
    years = range(2000, 2016)
    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(14, 9))
        for index, (column, country, name, note) in
enumerate(examples, start=1):
            country_data = df[df["country"] ==
country].sort_values("year")
            values =
country_data.set_index("year")[column].reindex(years)
            plt.subplot(2, 2, index)
            plt.bar(values.index, values, color="#497f72")
            plt.text(2000, values.max() * 0.08, note,
fontsize=10)
            plt.title(f"{name}: {column}", fontsize=11)
            plt.xlabel("年份")
            plt.ylabel("记录值")
            plt.xticks([2000, 2005, 2010, 2015])
            plt.xlim(1999.5, 2015.5)
            plt.ylim(bottom=0)
            plt.grid(axis="y", alpha=0.3)
            plt.gca().set_axisbelow(True)

        zero_rates: list[float] = []
        gdp_missing_rates: list[float] = []
        expenditure_missing_rates: list[float] = []
        # 各年保留行数不同，使用比例比较，而不是直接比较数量。
        for year in years:
            year_data = df[df["year"] == year]
            zero_rates.append((year_data["percentage_expenditure"
] == 0).sum() / len(year_data) * 100)
            gdp_missing_rates.append(year_data["gdp"].isna().sum(
) / len(year_data) * 100)
            expenditure_missing_rates.append(year_data["total_exp
enditure"].isna().sum() / len(year_data) * 100)
        plt.subplot(2, 2, 3)
        positions = np.arange(2000, 2016)
        plt.bar(positions - 0.25, zero_rates, width=0.25,
color="#c87c53", label="percentage_expenditure 为零")
        plt.bar(positions, gdp_missing_rates, width=0.25,
color="#638cac", label="gdp 缺失")
```

```
        plt.bar(positions + 0.25, expenditure_missing_rates,
width=0.25, color="#779a70", label="total_expenditure 缺失")
        plt.title("各年零值与缺失值占比", fontsize=11)
        plt.xlabel("年份")
        plt.ylabel("占当年保留记录的比例（%）")
        plt.xticks([2000, 2003, 2006, 2009, 2012, 2015])
        plt.xlim(1999.5, 2015.5)
        plt.ylim(0, 105)
        plt.legend(loc="upper left", fontsize=9)
        plt.grid(axis="y", alpha=0.3)
        plt.gca().set_axisbelow(True)

        values_2015 = df.loc[df["year"] == 2015,
"percentage_expenditure"].dropna()
        zero_count = int((values_2015 == 0).sum())
        nonzero_count = int((values_2015 != 0).sum())
        plt.subplot(2, 2, 4)
        counts = [zero_count, nonzero_count]
        plt.bar(["零值", "非零值"], counts, color=["#c87c53",
"#497f72"], width=0.6)
        for position, count in enumerate(counts):
            plt.text(position, count + 2, str(count),
ha="center")
        plt.title(f"2015 年 percentage_expenditure（共
{len(values_2015)}条非缺失记录）", fontsize=11)
        plt.ylabel("记录数量")
        plt.ylim(0, max(counts) * 1.12)
        plt.grid(axis="y", alpha=0.3)
        plt.gca().set_axisbelow(True)
        plt.suptitle("首轮清理后零值疑似缺失的证据")
        plt.tight_layout()
        plt.savefig(output_path, format="svg",
bbox_inches="tight")
        plt.close()

def plot_magnitude_scatter(df: pd.DataFrame, output_path: Path)
-> None:
    """用四张散点子图展示 gdp 和 population 的数量级突变。"""
    examples = [
        ("gdp", "Mauritius", "毛里求斯", "美元"),
        ("gdp", "Rwanda", "卢旺达", "美元"),
        ("population", "South Africa", "南非", "人"),
        ("population", "Hungary", "匈牙利", "人"),
    ]
    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(12, 8))
        for index, (column, country, name, unit) in
enumerate(examples, start=1):
            country_data = df[df["country"] ==
country].sort_values("year")
            plt.subplot(2, 2, index)
            plt.scatter(country_data["year"],
country_data[column], color="#638cac", s=35)
            plt.yscale("log")
            plt.title(f"{name}: {column}")
```

```
            plt.xlabel("年份")
            plt.ylabel(f"{unit}（对数刻度）")
            plt.xticks([2000, 2005, 2010, 2015])
            plt.xlim(1999.5, 2015.5)
            plt.grid(alpha=0.3)
        plt.suptitle("首轮清理后 gdp 与 population 的年度记录")
        plt.tight_layout()
        plt.savefig(output_path, format="svg",
bbox_inches="tight")
        plt.close()

if __name__ == "__main__":
    df, _ = drop_problem_rows(load_data())
    output_dir = Path(__file__).resolve().parent / "figures"
    output_dir.mkdir(parents=True, exist_ok=True)
    plot_bmi_lines(df, output_dir / "bmi 折线图.svg")
    plot_zero_values(df, output_dir / "零值疑似缺失.svg")
    plot_magnitude_scatter(df, output_dir / "数量级异常.svg")
    print(f"三张问题说明图已保存至：{output_dir}")

    df = clean_second_round(df)
    print("第二轮处理：已删除 gdp、population、bmi，指定字段的零值已
替换为缺失值。")
    print(f"剩余行数：{len(df)}，剩余字段数：{len(df.columns)}")
    missing_path = output_dir / "缺失值占比.svg"
    plot_missing_pie(df, missing_path)
    print(f"缺失值占比饼图已保存：{missing_path}")

    filled_df, fill_report = fill_missing_values(df)
    print("\n 缺失值处理统计：")
    print(fill_report.to_string(index=False))
    print(f"排除寿命缺失记录：{len(df) - len(filled_df)}行")
    print(f"补全后行数：{len(filled_df)}，剩余缺失值：
{int(filled_df.isna().sum().sum())}")
    fill_path = output_dir / "缺失值处理数量.svg"
    plot_missing_fill_counts(fill_report, fill_path)
    print(f"缺失值处理数量图已保存：{fill_path}")
    data_path = Path(__file__).resolve().with_name("寿命数据_补
全.csv")
    filled_df.to_csv(data_path, index=False, encoding="utf-8-
sig")
    print(f"补全后的分析数据已保存：{data_path}")
```